

Аудит @invoicebox/ui

Полный анализ библиотеки React-компонентов и план рефакторинга

Документация: 12 разделов · Июнь 2026

Содержание

1. Аудит @invoicebox/ui
2. Карта проекта — @invoicebox/ui
3. Базовая линия (baseline) — 2026-06-15
4. 02 — Отказоустойчивость (обработка ошибок)
5. 03 — Безопасность
6. 04 — Скорость и отзывчивость
7. 05 — Доступность (WCAG 2.1 AA)
8. 06 — Архитектура и линтер
9. 07 — Платформа (версии, сборка, CVE, кэш)
10. 08 — Тесты и CI
11. 09 — Наблюдаемость
12. 10 — План рефакторинга (волнами)

Аудит @invoicebox/ui

Полный аудит библиотеки React-компонентов, выполнен 2026-06-15 (ветка `projectAnalytics`), до каких-либо изменений кода. Документация ведётся в `audit/` (не в `docs/` — её затирает `release.sh: rm -rf docs`).

Документы

#	Файл	Содержание
00	00-PROJECT-MAP.md	Карта: стек, структура, API, сборка, ловушки
01	01-BASELINE-METRICS.md	Базовая линия метрик
02	02-resilience.md	Отказоустойчивость
03	03-security.md	Безопасность (XSS, redirect, CVE)
04	04-performance.md	Скорость, бандл, CLS
05	05-accessibility.md	Доступность WCAG 2.1 AA
06	06-architecture-lint.md	Архитектура и линтер
07	07-platform.md	Платформа, версии, релиз
08	08-tests.md	Тесты и CI
09	09-observability.md	Наблюдаемость
10	10-REFACTOR-PLAN.md	План рефакторинга волнами

Ключевой вывод

Это **прт-библиотека компонентов** (не приложение). Направления аудита адаптированы под специфику UI-kit (см. 00, раздел «Направления аудита для UI-библиотеки»): вместо «страничных» метрик — вес бандла, поставка, типобезопасность API и доступность.

Топ-риски: отсутствие CI и тестов критич. логики, 83 CVE в зависимостях (3 critical, почти все — dev-цепочка CRA/Storybook), близкое к нулю состояние доступности, непровалидированный `href` в `AppCardLink`, CJS-only сборка без tree-shaking, мёртвая зависимость `motion`. God-компонентов нет, секретов нет, `dangerouslySetInnerHTML` нет.

Код не менялся. Старт работ — после согласования плана (см. документ 10).

Карта проекта — @invoicebox/ui

Аудит от 2026-06-15. Базовая ветка: `develop` . Версия: `0.1.0-alpha.292` . Документ описывает фактическое состояние «как есть» перед рефакторингом. Код не менялся.

1. Что это

`@invoicebox/ui` — публикуемая в пмт библиотека React-компонентов (UI-kit) Invoicebox. Это не приложение: здесь нет роутинга, серверного рендеринга, глобального стора, страниц и пользовательских сценариев в классическом смысле. Поэтому ряд пунктов исходного ТЗ переинтерпретирован под специфику библиотеки (см. раздел «Применимость»).

- Потребитель: внешние/внутренние приложения Invoicebox, импортирующие компоненты.
- Витрина/документация: Storybook, публикуется на GitHub Pages (`ui.invoicebox.ru`).
- Лицензия: Apache 2.0.

2. Стек

Слой	Технология	Версия
UI	React (peer)	^18.2.0
Стили	styled-components (peer)	^6.1.8
Язык	TypeScript	4.9.5
Сборка библиотеки	Babel CLI (транспилиция, без бандлинга)	7.x
Типы	<code>tsc --emitDeclarationOnly</code>	—
Витрина	Storybook (react-webpack5)	^7.6.13
Тесты	Jest	^29.7.0
Линт	ESLint 8 + @typescript-eslint 5 + Prettier 3	—
Runtime (engines)	node >=16.0.0 (локально 16.20.2)	—

3. Структура

```
src/
  index.ts                – публичный API (единая точка экспорта)
  components/
    common/              – презентационные компоненты (Skeleton, Typography, Drawer,
                          Toast, QRCode, Calendar, Dropdown, Flag, AppCardLink ...)
    form/                – поля ввода (TextInput, Select, PhoneInput, DateInput,
                          DateTimeInput, CurrencyInput, Autocomplete, CodeInput ...)
    _helpers/            – вспомогательные компоненты для stories
    constants.ts, selectOptions.ts
  hooks/                 – useOutsideClick, useInputFocus, useUnupdatableHandler,
                          useLoadingSubmit, useMedia, useModal
  palette/               – система тем (PaletteProvider, useComponentPalette)
  validate/              – валидаторы РФ/СНГ (БИК, р/с, ОГРН, КПП, ИНН, website)
  breakpoints/           – точки адаптива
  utils/                 – transformColorByHsl
dist/                   – артефакт сборки (закоммичен; перезаписывается release.sh)
docs/                   – собранный Storybook (закоммичен; ВНИМАНИЕ: release.sh делает rm -rf
docs)
```

- ~38 компонентов, 270 .ts/.tsx файлов, 78 stories, 1 тест-файл.
- Самые крупные файлы: `form/PhoneInput` (341), `form/Select` (330), `form/DateTimeInput` (300), `form/Autocomplete` (269). God-компонентов нет.

4. Публичный API (поверхность)

Единый barrel-экспорт из `src/index.ts`: ~40 компонентов + типы, 9 хуков, `PaletteProvider` /палитра, `breakpoints`, `Validate`. Точечных entry-points нет — весь пакет тянется одним `main`:
`./dist/index.js` (см. перформанс: нет tree-shaking по умолчанию у потребителя через CJS).

5. Состояние (state)

Глобального стора нет. Состояние — локальное (`useState` / `useEffect`) внутри компонентов. Тема прокидывается через React Context (`PaletteProvider` → `useComponentPalette`). Это нормально для UI-kit.

6. Сборка / тесты / релиз

Команда	Действие
<code>npm run storybook</code>	дев-витрина на :6006
<code>npm test (jest)</code>	прогон тестов
<code>npm run release (./release.sh)</code>	сборка dist + Storybook, бамп версии, merge в <code>main</code> , <code>npm publish</code>

`release.sh` (ловушки см. ниже): транпиляция Babel → `dist`, типы → `tsc`, Storybook → `docs`, CNAME, коммит, бамп `prerelease --preid=alpha`, merge `develop`→`main`, `npm publish`.

7. Переменные окружения

Не используются. `.npmrc` фиксирует публичный реестр npm. Секретов в коде нет (скан по `api_key/secret/token/password` — чисто; единственное совпадение — `autoComplete="new-password"`, легитимно).

8. Известные ловушки (traps)

- `release.sh` делает `rm -rf dist docs` — любая ручная документация в `docs/` будет уничтожена при релизе. Поэтому аудит лежит в `audit/`. ⚠️
- `dist/` закоммичен и при этом в `.gitignore` — рассинхрон; jest без `testPathIgnorePatterns` гоняет тесты дважды (из `src` и из `dist`).
- Релиз пушит в `main` и публикует в npm из локальной машины — нет CI-гейта, нет проверки тестов/линта перед публикацией. Высокий риск выката битого пакета.
- `engines.node >=16`, но Storybook 7 и часть инструментов требуют node 18+. Заявленная совместимость недостоверна.
- CJS-only сборка (`modules: commonjs`) без `sideEffects` / ESM — потребитель не получает tree-shaking; тянет весь пакет.
- Тяжёлая `motion` (`framer-motion`-преемник) в `dependencies` не используется — мёртвая зависимость в проде.

9. Направления аудита для UI-библиотеки

T3 изначально написано под веб-приложение. Поскольку это публикуемая библиотека компонентов, направления адаптированы под её специфику: вместо «страничных» метрик (маршруты, стор, Lighthouse, OAuth/CSRF, серверные редиректы) — то, что реально влияет на потребителей пакета и на качество самого UI-kit.

Направление аудита	Что это значит для библиотеки	Документ
Безопасность	непровалидированный <code>href</code> / <code>target</code> (<code>AppCardLink</code>), regex-валидаторы (<code>Validate.website</code> , ReDoS), CVE в зависимостях, отсутствие секретов	03
Отказоустойчивость	компонент не должен ронять дерево потребителя: <code>ErrorBoundary</code> , <code>guard</code> -парсинг, безопасный сброс состояний	02
Размер и поставка	вес <code>dist</code> , CJS-only без tree-shaking, <code>exports</code> -map, мёртвые зависимости, бюджеты <code>size-limit</code>	04
Визуальная стабильность	CLS и мерцание на стороне компонентов (<code>Skeleton/Drawer</code>), <code>prefers-reduced-motion</code>	04
Доступность (WCAG AA)	клавиатура/ARIA/фокус-трав в BCEX инпутах и модалках — тиражируется во все приложения-потребители	05
Качество публичного API	типобезопасность экспортов (без <code>any</code>), консистентность пропов, отсутствие <code>god</code> -компонентов	06
Платформа и сборка	актуальность TS/ESLint/Storybook, версия node-engine, воспроизводимость и безопасность релиза/публикации в npm	07
Тесты и CI	покрытие критичной логики (валидаторы реквизитов/денег), компонентные тесты, гейт в CI	08
Наблюдаемость	управляемое логирование, <code>onError</code> -хуки для потребителя, CHANGELOG/ версия в рантайме	09

Прямые замеры Lighthouse/Web Vitals/SSR к npm-пакету неприменимы (нет страницы) — их роль выполняют размер бандла, `size-limit` и a11y-проверки на Storybook.

10. Документы аудита

- `01-BASELINE-METRICS.md` — стартовые показатели (база сравнения)
- `02-resilience.md` — отказоустойчивость / обработка ошибок
- `03-security.md` — безопасность
- `04-performance.md` — скорость и отзывчивость
- `05-accessibility.md` — доступность WCAG 2.1 AA
- `06-architecture-lint.md` — архитектура и линтер
- `07-platform.md` — платформа (версии, сборка, CVE, кэш)
- `08-tests.md` — тесты и CI
- `09-observability.md` — наблюдаемость
- `10-REFACTOR-PLAN.md` — план рефакторинга волнами

Базовая линия (baseline) — 2026-06-15

Зафиксировано до изменений. Служит точкой сравнения после каждой волны рефакторинга.

Замер: окружение

- node (локально): v16.20.2
- TypeScript: 4.9.5
- Ветка: `projectAnalytics`, базовая `develop`.

Тесты

Метрика	Значение
Тест-файлов	1 (<code>src/validate/utils/websiteValidate.test.ts</code>)
Тестов (assertions suites)	4 теста, проходят
Test Suites при <code>jest</code>	2 (дубль из <code>dist/</code> — нет <code>testPathIgnorePatterns</code>)
Coverage	не настроен (нет <code>--coverage</code> , нет порогов)
CI	отсутствует (<code>.github/workflows</code> нет)

Зависимости / уязвимости (`npm audit`)

Severity	Кол-во
critical	3
high	31
moderate	37
low	12
total	83

Деревья: prod 119, dev 2146, peer 657 (итого 2267 пакетов). **Критические:** `form-data` (unsafe random boundary), `handlebars` (JS injection), `shell-quote` (escape newlines). Подавляющее большинство high/critical — в dev-цепочке Storybook 7 / `react-scripts` / `@svgr` / `workbox` (транзитивно). Подробно — `03-security.md` / `07-platform.md`.

Размер артефакта

Метрика	Значение
<code>dist/</code> (транспирированный, без минификации)	2.4 MB
Формат	CommonJS, без бандлинга/tree-shaking/минификации
<code>package-lock.json</code>	2.39 MB
Мёртвая prod-зависимость	<code>motion</code> ^12.38.0 (0 использований в <code>src</code>)

Для библиотеки точный «размер бандла у потребителя» зависит от его сборщика. Зафиксированы: размер `dist` , факт CJS-only и отсутствие `sideEffects` . После рефакторинга добавить измерение

per-component через `size-limit / esbuild --metafile` (см. `04-performance.md`).

Код-метрики (статически)

Метрика	Значение
Файлов <code>.ts/.tsx</code> (src)	270
Компонентов	~38
Stories	78
Крупнейший файл	341 LOC (PhoneInput)
<code>any</code> / небезоп. assertions (<code>as any</code> , <code>as unknown</code>)	11 вхождений
<code>@ts-ignore</code> / <code>@ts-expect-error</code>	0
<code>eslint-disable</code>	22
<code>console.*</code>	3
<code>try/catch</code> блоков	0
ErrorBoundary	0
<code>aria-*</code> атрибутов	0
<code>role=</code>	0
<code>onKeyDown/Up/Press</code>	2
<code>prefers-reduced-motion</code>	0
<code>dangerouslySetInnerHTML</code>	0
Секреты в коде	0

Сводный «светофор»

Направление	Оценка
Отказоустойчивость	нет обработки ошибок вовсе
Безопасность (код)	точечные (<code>href</code> , <code>regex</code>); секретов нет
Безопасность (deps)	83 уязвимости, 3 critical
Производительность	CJS-only, мёртвый <code>motion</code> , нет <code>tree-shaking</code>
Доступность	практически отсутствует
Архитектура/типы	в целом, точечные <code>any</code> / <code>disable</code>
Платформа	устаревшие TS/ESLint/node-engine, ручной релиз
Тесты	1 файл, нет CI, нет coverage
Наблюдаемость	нет единой стратегии логирования/ошибок

02 — Отказоустойчивость (обработка ошибок)

Принцип для UI-kit: компонент не должен «ронять» дерево приложения-потребителя. Сейчас обработка ошибок отсутствует как класс — `try/catch` = 0, `ErrorBoundary` = 0.

R-1. Нет ни одного `ErrorBoundary` в библиотеке

- **Где:** весь `src` (0 совпадений `componentDidCatch` / `getDerivedStateFromError`).
- **Серьёзность:** medium (для библиотеки — спорно: `ErrorBoundary` чаще ответственность приложения, но «составные» компоненты с асинхронным контентом стоит защищать).
- **Описание:** если внутренний рендер падает (например, кривые данные в `Autocomplete` / `Select` / `Calendar`), исключение всплывает в приложение-потребитель и валит весь экран.
- **Решение:** экспортировать опциональный `<ErrorBoundary>` и оборачивать им «тяжёлые» составные компоненты (`Autocomplete`, `DateTimeInput`, `Calendar`) изнутри с безопасным fallback. Не навязывать — давать проп `renderError`.
- **Эффект:** локализация сбоя одним компонентом вместо падения всей страницы.

R-2. Парсинг/числовые операции без защиты от мусора

- **Где:** `src/validate/index.ts` (`parseInt(value.slice...)`, `Number(bikRs[i])`), `form/CurrencyInput/*`, `form/PositiveIntegerInput`, `form/DateInput` / `DateTimeInput`.
- **Серьёзность:** medium.
- **Описание:** `parseInt` / `Number` на неожиданном вводе дают `NaN`, который тихо протекает в контрольные суммы и расчёты дат — невалидный результат без явной ошибки.
- **Решение:** guard-проверки (`Number.isFinite`), ранний возврат `createError`, нормализация ввода до вычислений. Покрывать тестами edge-cases.
- **Эффект:** предсказуемые ошибки валидации вместо «тихого» `NaN`.

R-3. `useEffect` с таймерами/подписками — проверить очистку

- **Где:** `common/Drawer/index.tsx` (`setTimeout 300мс` — очистка есть ) , `hooks/useOutsideClick`, `hooks/useMedia`, компоненты с `addEventListener`.
- **Серьёзность:** low.
- **Описание:** часть эффектов корректна; нужен сплошной аудит на утечки слушателей/ таймеров при размонтировании.
- **Решение:** убедиться, что каждый `addEventListener` / `setTimeout` / `setInterval` имеет `cleanup`; добавить ESLint `react-hooks/exhaustive-deps` (сейчас только recommended).
- **Эффект:** отсутствие утечек памяти и «фантомных» обработчиков.

R-4. Асинхронные обработчики без `.catch`

- **Где:** `hooks/useLoadingSubmit`, компоненты с `onChange` / `onSubmit`, прокидывающие промисы потребителя.
- **Серьёзность:** medium.
- **Описание:** линт уже включает `no-floating-promises` / `no-misused-promises` (хорошо), но в рантайме отклонённый промис из колбэка потребителя может оставить спиннер навсегда.
- **Решение:** в `useLoadingSubmit` оборачивать вызов в `try/finally`, гарантируя сброс `isLoading` даже при `reject`; задокументировать контракт.

- **Эффект:** UI не «залипает» в состоянии загрузки при ошибке сабмита.

R-5. Нет деградации при отсутствии `PaletteProvider`

- **Где:** `palette/useComponentPalette.ts`, `usePalette.ts`.
 - **Серьёзность:** low/medium.
 - **Описание:** если потребитель забыл обернуть в `PaletteProvider`, доступ к палитре может бросить/вернуть `undefined` → падение стилей.
 - **Решение:** дефолтная палитра как fallback + понятный `console.warn` (dev-only).
 - **Эффект:** компоненты рендерятся с дефолтной темой, а не падают.
-

Приоритет блока: R-2 и R-4 (medium) — в волну 1; R-1/R-5 — волна 2; R-3 — фоном.

03 — Безопасность

Модель угроз для UI-kit: компонент получает данные/URL от приложения-потребителя, часть которых может происходить из пользовательского ввода. Библиотека обязана не создавать каналов XSS/open-redirect и не тянуть уязвимые зависимости в прод.

Хорошие новости: `dangerouslySetInnerHTML` = 0, секретов/ключей/токенов в коде = 0 (единственное совпадение — `autoComplete="new-password"`, легитимно). Сети/OAuth/CSRF в библиотеке нет — эти пункты ТЗ неприменимы.

S-1. `AppCardLink`: непровалидированный `href` + `target="_blank"` без `rel`

- **Где:** `src/components/common/AppCardLink/index.tsx:26` — `<S.Wrapper href={href} target={'_blank'}>`.
- **Серьёзность:** high.
- **Описание:** два дефекта в одной строке:
- `href` рендерится «как есть». Если потребитель передаст `javascript:...` (или `data:`) URL из пользовательского ввода — это XSS при клике (DOM-based).
- `target="_blank"` без `rel="noopener noreferrer"` — reverse tabnabbing и утечка `Referer`. (Современные браузеры по умолчанию ставят `noopener` для `_blank`, но полагаться на это нельзя — это поведение и старые webview.)
- **Решение:**
- Валидировать схему: разрешать только `https: / http:` (и относительные), отклонять `javascript: / data: / vbscript:`. Реализовать `isSafeUrl(url)` в `utils`.
- Жёстко проставлять `rel="noopener noreferrer"` для всех `target="_blank"`.
- **Эффект:** закрыт DOM-XSS и tabnabbing на самой частой ссылочной точке библиотеки.
- **Best practice:** OWASP DOM XSS Prevention; React docs — «only `http / https` URLs».

S-2. `Validate.website` — небезопасный для повторного использования как «санитайзер»

- **Где:** `src/validate/index.ts` (метод `website`, экспортируется в публичный API).
- **Серьёзность:** medium.
- **Описание:**
- Регулярка не закорена справа (`^(...)?` без `$`) — валидирует «начинается с», а не «является» URL. Для проверки формата поля терпимо, но опасно, если значение потом используется как `href`.
- Схема опциональна (`(?:https?:\\\/)?`) — `abc.com` проходит без протокола; при построении ссылки потребитель обязан сам добавить `https://` (иначе относительный путь → open-redirect внутри своего домена).
- Вложенные квантификаторы (`{0,61}...+ ...`) — потенциальный ReDoS на спец-входе.
- **Решение:** добавить правый якорь `$`; в JSDoc явно указать «валидация формата, НЕ санитизация для href»; заменить на безопасный парсинг через `new URL()` где возможно; прогнать regex через ReDoS-чек (например, `recheck / safe-regex`).
- **Эффект:** валидатор перестаёт быть источником ложной уверенности и ReDoS-вектором.

S-3. 83 уязвимости в зависимостях (3 critical, 31 high)

- **Где:** `npm audit` — преимущественно dev-цепочка `Storybook 7 / react-scripts / @svgr/* / workbox-* / rollup-plugin-terser`.

- **Серьёзность:** critical (формально), фактический риск — в основном dev-time.
- **Описание:** critical: `form-data`, `handlebars`, `shell-quote`. Большинство — не попадает в опубликованный `dist` (только `peer react / styled-components` идут в прод), но влияет на безопасность сборочной машины и доверие к пакету.
- **Решение:** см. `07-platform.md` (апгрейд Storybook 8/9, отказ от `react-scripts / @storybook/preset-create-react-app`, который и тянет `workbox / svgr`). Зафиксировать `npm audit` в CI с порогом.
- **Эффект:** снижение CVE-поверхности на десятки пунктов, чистый `audit` как гейт.

S-4. CSP / защитные заголовки — для Storybook на GitHub Pages

- **Где:** публикация `docs/` (Storybook) на `ui.invoicebox.ru`.
- **Серьёзность:** low.
- **Описание:** статическая витрина без CSP/ `X-Content-Type-Options` / `Referrer-Policy`. Для самой `npm`-библиотеки CSP неприменим (заголовки задаёт приложение-потребитель).
- **Решение:** добавить мета-CSP в `.storybook/preview-head.html` или заголовки на уровне хостинга; в README библиотеки — рекомендации по CSP для потребителей.
- **Эффект:** базовая защита витрины, явный гайд для интеграторов.

S-5. `hex-to-rgba` и ручные строковые операции с цветом

- **Где:** 17 файлов используют `hex-to-rgba`; `utils/transformColorByHsl.ts`.
- **Серьёзность:** low.
- **Описание:** цвет приходит из палитры потребителя; некорректный ввод не валидируется — не XSS (вставляется в CSS через `styled-components`, который экранирует), но возможен «битый» стиль.
- **Решение:** валидация/нормализация цвета на входе в `PaletteProvider`.
- **Эффект:** устойчивость темизации к некорректным значениям.

Приоритет блока: S-1 (high) — волна 1; S-2/S-3 — волна 1–2; S-4/S-5 — волна 3.

04 — Скорость и отзывчивость

Для библиотеки «производительность» = (а) размер кода, доезжающего до бандла потребителя, (б) tree-shakability, (в) отсутствие тяжёлого в критическом пути, (г) визуальная стабильность компонентов (CLS, мерцание скелетонов).

P-1. CJS-only сборка без tree-shaking

- **Где:** `babel.config.json` (`"modules": "commonjs"`), `package.json` (`main: ./dist/index.js`, нет `module / exports / sideEffects`).
- **Серьёзность:** high.
- **Описание:** пакет публикуется только в CommonJS, единым barrel-ом. Потребитель, импортирующий один `TextInput`, получает в бандл весь граф (`react-calendar`, `qrcode.react`, `react-modal-sheet`, маски и т.д.), т.к. CJS не шейкается.
- **Решение:** добавить ESM-сборку и `"exports"` map с подпутями (`@invoicebox/ui/TextInput`), проставить `"sideEffects": false` (с осторожностью к `styled-components`). Перейти на бандлер библиотек (`tsup / rollup / vite lib`).
- **Эффект:** кратное снижение размера у потребителей, реальный tree-shaking.

P-2. Мёртвая зависимость `motion` (~framer-motion) в проде

- **Где:** `package.json dependencies motion ^12.38.0`; использований в `src = 0`.
- **Серьёзность:** high (по соотношению вес/польза).
- **Описание:** одна из самых тяжёлых анимационных библиотек объявлена прод-зависимостью и не используется. Тянется в граф установки потребителя.
- **Решение:** удалить из `dependencies` (после подтверждения, что не нужна).
- **Эффект:** минус крупная зависимость из дерева, чище установка.

P-3. Нет измерения размера бандла `per-component`

- **Где:** процесс сборки.
- **Серьёзность:** medium.
- **Описание:** размер `dist` 2.4 MB (транспилиция без минификации), но реального `per-export` бюджета нет — регрессии веса не отлавливаются.
- **Решение:** `size-limit` с бюджетами на ключевые экспорты в CI; `esbuild --metafile` для анализа графа.
- **Эффект:** контроль регрессий размера на каждом PR.

P-4. Тяжёлые компоненты в общем графе (Calendar/QRCode/масками)

- **Где:** `react-calendar` (7 файлов), `qrcode.react`, `@mona-health/react-input-mask`, `react-custom-scrollbars-2`.
- **Серьёзность:** medium.
- **Описание:** редко используемые тяжёлые компоненты (QRCode, Calendar) попадают в граф даже когда не нужны (следствие P-1).
- **Решение:** после P-1 — изоляция через подпути-exports + опциональный `React.lazy` для демонстраций; вынести в `peer/optional` где уместно.
- **Эффект:** критический путь форм не тянет календарь/QR.

P-5. CLS / мерцание — Skeleton и Drawer

- **Где:** `common/Skeleton`, `common/Drawer/index.tsx` (задержка размонтирования 300мс), загрузочные состояния форм.
- **Серьёзность:** medium.
- **Описание:** скелетоны без фиксированных размеров вызывают сдвиг макета (CLS) при подмене на контент; множественные независимые скелетоны → «скелетон-шторм» и мерцание. Нужен единый загрузчик с задержкой появления (anti-flicker) и резервированием места.
- **Решение:** Skeleton с обязательными `width/height`; глобальная политика «единый загрузчик с delay 150–200мс и min-display» против мерцания; зарезервированные размеры у всех плейсхолдеров.
- **Эффект:** CLS→~0, нет мельтешения при быстрых загрузках.

P-6. Анимации без `prefers-reduced-motion`

- **Где:** `prefers-reduced-motion` = 0 совпадений; анимации в Drawer/Toast/спиннерах.
- **Серьёзность:** medium (пересечение с a11y, см. 05).
- **Описание:** анимации всегда включены — нагрузка на слабые устройства + проблема доступности.
- **Решение:** общий хук/CSS-обёртка, отключающая анимации при `prefers-reduced-motion: reduce`.
- **Эффект:** меньше работы рендера на слабых девайсах, выполнение WCAG 2.3.3.

Приоритет блока: P-1 + P-2 (high) — волна 1 (измеримый эффект); P-3/P-5 — волна 2; P-4/P-6 — волна 3.

05 — Доступность (WCAG 2.1 AA)

Самый слабый блок. По всему `src : aria-* = 0, role = 0`, обработчиков клавиатуры (`onKeyDown/Up/Press`) = 2, `prefers-reduced-motion = 0`. Для библиотеки **форм-инпутов** это критично — недоступность тиражируется во все приложения-потребители.

А-1. Кастомные интерактивные компоненты не работают с клавиатуры

- **Где:** `form/Select`, `common/Dropdown`, `form/Autocomplete`, `form/Toggle`, `form/DateInput / DateTimeInput`, `form/CodeInput`, `common/Drawer`.
- **Серьёзность:** **critical**.
- **Описание:** `dropdown/select/autocomplete` построены на `div`-ах без клавиатурной навигации (стрелки/Enter/Esc/Home/End), без `tabindex`, без управления фокусом. Пользователь клавиатуры/скринридера не может ими пользоваться. Нарушение WCAG 2.1.1 (Keyboard), 4.1.2 (Name/Role/Value).
- **Решение:** реализовать паттерны WAI-ARIA Authoring Practices:
- `Select/Combobox` → `role="combobox"` + `aria-expanded / aria-controls / aria-activedescendant`, список `role="listbox"`, опции `role="option"`, навигация стрелками, Enter/Esc.
- `Toggle` → нативный `<input type="checkbox/switch">` или `role="switch"` + `aria-checked`, активация Space/Enter.
- Рассмотреть Radix UI / React Aria как базу примитивов (доступность «из коробки»).
- **Эффект:** полноценная работа с клавиатуры и скринридерами во всех потребителях.

А-2. Поля без программной связи `label↔control` и сообщений об ошибке

- **Где:** `form/InputLabel`, `form/TextInput`, `form/PureInput`, все поля.
- **Серьёзность:** **high**.
- **Описание:** нет гарантии `htmlFor / id` связи, ошибки валидации не объявляются (`aria-invalid`, `aria-describedby` на текст ошибки). Скринридер не сообщает причину ошибки. WCAG 1.3.1, 3.3.1, 3.3.2, 4.1.3.
- **Решение:** `InputLabel` генерирует/прокидывает `id`, поле получает `aria-invalid`, ошибка — `role="alert"` + связь через `aria-describedby`.
- **Эффект:** доступная валидация форм, корректное озвучивание ошибок.

А-3. Модалки/Drawer без фокус-трапа и возврата фокуса

- **Где:** `common/Drawer/index.tsx` (на `react-modal-sheet`), `hooks/useModal`.
- **Серьёзность:** **high**.
- **Описание:** нет гарантии фокус-трапа внутри открытого Drawer, возврата фокуса на триггер при закрытии, `role="dialog" / aria-modal / aria-labelledby`, закрытия по Esc. WCAG 2.1.2 (No keyboard trap наоборот — нужен управляемый трап), 2.4.3 (Focus Order).
- **Решение:** `focus-trap` (`focus-trap-react` или примитив диалога Radix), возврат фокуса, `aria-modal="true"` + `aria-labelledby` на заголовок, Esc-закрытие, блокировка фона.
- **Эффект:** доступные модалные сценарии, корректный порядок фокуса.

А-4. Контраст и размеры тач-таргетов не верифицированы

- **Где:** `palette/constants.ts`, `Typography`, кнопки/чипы/иконки.

- **Серьёзность:** medium.
- **Описание:** нет проверки контраста текста/фона ($\geq 4.5:1$ для обычного текста) и минимального размера интерактивных целей ($\approx 44 \times 44 \text{px}$). WCAG 1.4.3, 2.5.5.
- **Решение:** прогнать палитру через контраст-чекер, задокументировать допустимые пары; гарантировать min-таргеты в стилях кнопок/иконок-кнопок.
- **Эффект:** соответствие AA по контрасту и удобство на тач-устройствах.

A-5. Иконки/SVG без текстовых альтернатив

- **Где:** `common/InvoiceboxLogo`, `Flag`, `WarningIcon`, `Arrow`, `AppCardLink/components/*` (`GooglePlay/Apk/RuStore/AppGallery`), `QRCode`.
- **Серьёзность:** medium.
- **Описание:** значимые SVG без `role="img" / aria-label` или `aria-hidden` для декоративных. Ссылки-картинки магазинов без доступного имени. WCAG 1.1.1.
- **Решение:** значимым — `aria-label / <title>`, декоративным — `aria-hidden="true"`; `AppCardLink` — доступное имя ссылки.
- **Эффект:** скринридер корректно описывает графику и ссылки.

A-6. Нет учёта `prefers-reduced-motion` и масштабирования

- **Где:** анимации (`Drawer/Toast/спиннеры`), вёрстка в `px`.
- **Серьёзность:** medium.
- **Описание:** анимации не отключаются по `reduce`; жёсткие `px` мешают масштабированию до 200% (WCAG 1.4.4, 1.4.10 Reflow, 2.3.3 Animation from Interactions).
- **Решение:** `media-query prefers-reduced-motion`; перевод критичных размеров на `rem`; проверка reflow при zoom 200%.
- **Эффект:** комфорт для чувствительных к движению и слабовидящих пользователей.

Приоритет блока: A-1 (critical), A-2/A-3 (high) — волны 1–2 (по компонентам); A-4/A-5/A-6 — волна 3.

Рекомендация: подключить `eslint-plugin-jsx-a11y` и `@storybook/addon-a11y` уже в волне 1 как измеримый гейт.

06 — Архитектура и линтер

В целом структура здоровая: понятное деление `common / form / hooks / palette / validate`, единый `barrel-API`, `god`-компонентов нет (макс 341 LOC). Проблемы точечные.

AR-1. `any` и небезопасные `assertions`

- **Где:** 11 вхождений `any / as any / as unknown` по `src` (формы/автокомплит/палитра).
- **Серьёзность:** medium.
- **Описание:** дыры в типобезопасности публичного API библиотеки → ошибки протекают к потребителю без подсказок типов. Плюс — `@ts-ignore = 0` (хорошо).
- **Решение:** заменить `any` на дженерики/ `unknown` + сужение; запретить правилами `@typescript-eslint/no-explicit-any` (error) и `no-unsafe-*` после апгрейда tooling.
- **Эффект:** надёжные типы на поверхности пакета, меньше runtime-сюрпризов.

AR-2. Нет ограничения размера файла/сложности в линте

- **Где:** `.eslintrc.json` — нет `max-lines / max-lines-per-function / complexity`.
- **Серьёзность:** low.
- **Описание:** ничто не сдерживает рост файлов (`PhoneInput/Select` уже 330–341 LOC).
- **Решение:** добавить `max-lines: [warn, 300]`, `max-lines-per-function`, `complexity` как `warn`; крупные компоненты декомпозировать на под-компоненты/хуки.
- **Эффект:** предсказуемый размер модулей, ранний сигнал о росте сложности.

AR-3. Лишние ререндеры — необёрнутые объекты/мапы в рендере

- **Где:** напр. `common/AppCardLink/index.tsx` (`appCardMap` создаётся на каждый рендер), обработчики/объекты-литералы в JSX по формам.
- **Серьёзность:** low/medium.
- **Описание:** объекты/функции пересоздаются каждый рендер → лишние ререндеры дочерних мемоизированных компонентов.
- **Решение:** выносить статические мапы за компонент; `useMemo / useCallback` для стабильных ссылок там, где есть мемо-потомки. Не мемоизировать вслепую.
- **Эффект:** меньше лишних ререндеров в формах.

AR-4. 22 `eslint-disable` — точно скрытый долг

- **Где:** 22 вхождения по `src` (в т.ч. `src/index.ts` — `no-var-requires / no-undef` для `shim-object.entries/fromentries`).
- **Серьёзность:** low.
- **Описание:** часть оправдана (полифилы), часть может маскировать реальные проблемы.
- **Решение:** ревизия каждого `disable`; `shim`-полифилы вынести/заменить нативными `Object.entries/fromEntries` (доступны в target ES2022) — снять зависимости `object.entries / object.fromentries` целиком.
- **Эффект:** минус 2 зависимости + чище линт.

AR-5. Консистентность экспорта/именования

- **Где:** `src/index.ts` — смесь `export type { TProps as ... };` имена палитр (`drawer`), пропсы `is*` - флаги.
- **Серьёзность:** low.
- **Описание:** конвенции в целом единые, но `TProps` -переименования и булевы `is*` / `should*` стоит зафиксировать в гайдлайне для предсказуемости API.
- **Решение:** короткий CONTRIBUTING/стайлгайд по именованию пропсов и экспортов.
- **Эффект:** предсказуемый, документированный публичный API.

AR-6. `prop-types` в `devDeps` у TS-библиотеки

- **Где:** `package.json` `devDeps` `prop-types`; `react/prop-types` выключен в линте.
 - **Серьёзность:** low.
 - **Описание:** рудимент — типизация уже на TS; runtime `prop-types` в `src` не нужны (0).
 - **Решение:** удалить `prop-types`, если не используется stories/helpers.
 - **Эффект:** чище зависимости.
-

Приоритет блока: AR-1 — волна 2; AR-2/AR-4/AR-6 — волна 1 (дёшево, измеримо); AR-3/AR-5 — волна 3.

07 — Платформа (версии, сборка, CVE, кэш)

PL-1. Устаревший инструментарий (TS / ESLint / typescript-eslint)

- **Где:** `package.json` — TypeScript 4.9.5, ESLint 8.x, `@typescript-eslint/* 5.x`.
- **Серьёзность:** medium.
- **Описание:** TS 4.9 (текущие — 5.x), ESLint 8 (EOL), typescript-eslint 5 (старая мажорка) — нет современных правил (no-unsafe-*), типов и багфиксов.
- **Решение:** апгрейд до TS 5.x, ESLint 9 (flat config) или хотя бы 8 latest, typescript-eslint 8. Прогон автофиксов, актуализация конфигов.
- **Эффект:** современные проверки типов/линта, основа для AR-1.

PL-2. Storybook 7 тянет уязвимую react-scripts /CRA-цепочку (источник CVE)

- **Где:** devDeps `@storybook/preset-create-react-app`, `@storybook/* 7.6`, транзитивно `react-scripts` → `@svgr/*`, `workbox-*`, `rollup-plugin-terser`, `svgo`.
- **Серьёзность:** high.
- **Описание:** именно CRA-пресет тянет основную массу high/critical из `npm audit`. CRA задепрекейчен.
- **Решение:** мигрировать на Storybook 8/9 на Vite-билдере, выкинуть `preset-create-react-app / react-scripts`. Это снимет десятки CVE разом.
- **Эффект:** резкое падение `npm audit`, быстрее dev-сборка витрины.

PL-3. Заявленный engines.node >=16 недостоверен

- **Где:** `package.json` "engines": { "node": ">=16.0.0" }; локально node 16.20.2.
- **Серьёзность:** medium.
- **Описание:** Storybook 7+ и современный tooling требуют node 18+; node 16 — EOL (с 2023). Заявленная совместимость вводит в заблуждение и ломает воспроизводимость.
- **Решение:** поднять до `>=18` (лучше `>=20`); добавить `.nvmrc / engine-strict`; зафиксировать версию в CI-матрице.
- **Эффект:** воспроизводимая сборка, поддерживаемый рантайм.

PL-4. Сборка не воспроизводима и без бандлинга

- **Где:** `release.sh` (Babel CLI транспиляция, без минификации/бандла), `dist/` закоммичен в репозиторий.
- **Серьёзность:** medium.
- **Описание:** артефакт собирается ad-hoc, коммитится в git (раздувает историю), нет ESM/минификации (см. P-1). Babel-транспиляция «как есть».
- **Решение:** перейти на `tsup / rollup` (ESM+CJS+минификация+ exports), убрать `dist` из git (`.npmignore / files` уже ограничивает публикацию — но коммит лишний), собирать в CI.
- **Эффект:** воспроизводимый, минифицированный, tree-shakable артефакт.

PL-5. Релиз вручную с локальной машины, без гейта

- **Где:** `release.sh` — `git checkout main && merge develop && npm publish` локально.
- **Серьёзность:** high (операционный риск).

- **Описание:** публикация в прт и пуш в `main` идут с машины разработчика без прогона тестов/линта/audit, без 2FA-гейта, без provenance. Риск выката битого/непроверенного пакета и человеческой ошибки в git.
- **Решение:** CI-релиз (GitHub Actions) с теров: lint+test+audit+build → `npm publish` с `--provenance`, OIDC-токеном вместо локального; `main` под branch protection.
- **Эффект:** надёжный, проверяемый, аудируемый релиз.

PL-6. Кэширование статики витрины

- **Где:** Storybook на GitHub Pages (`docs/` , `CNAME ui.invoicebox.ru`).
- **Серьёзность:** low.
- **Описание:** GitHub Pages даёт ограниченный контроль заголовков кэша; хешированные ассеты Storybook кэшируются, но `index.html` стоит держать `no-cache` .
- **Решение:** при переходе на собственный хостинг — immutable-кэш для хешированных ассетов, `no-cache` для html. На Pages — оставить как есть (вне контроля).
- **Эффект:** корректная инвалидция витрины (низкий приоритет).

Приоритет блока: PL-2 + PL-5 (high) — волна 1; PL-1/PL-3/PL-4 — волна 2; PL-6 — фоном.

08 — Тесты и CI

Текущее покрытие близко к нулю: **1 тест-файл, 4 теста** (`websiteValidate.test.ts`), на ~38 компонентов и весь модуль `validate` . CI нет.

T-1. Нет CI вообще

- **Где:** `.github/workflows` отсутствует.
- **Серьёзность:** `high`.
- **Описание:** ни тесты, ни линт, ни `audit` , ни сборка не проверяются автоматически. Любой PR может слить регрессию; релиз идёт без гейта (см. PL-5).
- **Решение:** GitHub Actions: на PR — `install → lint → typecheck → test --coverage → build → npm audit` (порог) ; на тег — релиз. Матрица node 18/20.
- **Эффект:** автоматический контроль качества каждого изменения.

T-2. Критический путь без тестов

- **Где:** `validate/utils/vatNumberValidateHelpers.ts` (ИНН/ОГРН/БИК/р-с СНГ), `validate/index.ts` (контрольные суммы `settlementAccount/correspondentAccount/ОГРН`), `form/PhoneInput/phoneInputLogic.ts` , `CurrencyInput` , `DateInput / DateTimeInput` .
- **Серьёзность:** `high`.
- **Описание:** бизнес-критичная логика (банковские реквизиты, контрольные числа, телефоны, деньги, даты) почти не покрыта. Ошибка здесь = неверная валидация платёжных данных у всех потребителей.
- **Решение:** unit-тесты на все валидаторы (включая edge: пустые/мусорные/граничные длины, NaN), на `phone/currency/date`-логику. Это самый высокий ROI.
- **Эффект:** защита денежно-критичной логики, ловля регрессий.

T-3. Нет компонентных/integration-тестов

- **Где:** компоненты `form/*` , `common/Drawer` , `Select` , `Autocomplete` .
- **Серьёзность:** `medium`.
- **Описание:** нет проверки рендера, взаимодействий, доступности.
- **Решение:** React Testing Library + `jest-dom` + `user-event` ; `jest-axe` для a11y- регрессий (синергия с 05); Storybook play-functions (`@storybook/test` уже в devDeps, но не используется) для interaction-тестов.
- **Эффект:** покрытие поведения и доступности ключевых компонентов.

T-4. Jest гоняет тесты дважды (из `dist/`)

- **Где:** `jest` без конфига → подхватывает `dist/validate/websiteValidate.test.js` .
- **Серьёзность:** `low`.
- **Описание:** дублирующий прогон скомпилированной копии; зашумляет вывод, маскирует расхождения `src/dist`.
- **Решение:** `jest.config` с `testPathIgnorePatterns: ['/dist/', '/node_modules/']` , `coverageThreshold` , окружение `jsdom` для компонентов.
- **Эффект:** чистый, детерминированный прогон + пороги покрытия.

T-5. Нет e2e критического пути

- **Где:** —
 - **Серьёзность:** low (для библиотеки).
 - **Описание:** полноценный e2e в UI-kit избыточен; роль e2e выполняют Storybook interaction-тесты и визуальная регрессия.
 - **Решение:** опционально — визуальная регрессия (Chromatic/Playwright-screenshots) по stories; в CI на PR.
 - **Эффект:** ловля визуальных регрессий без тяжёлого e2e-стека.
-

Целевые пороги (предложение): старт — coverage по `validate/` $\geq 80\%$; общий $\geq 50\%$ к концу волны 2, $\geq 70\%$ к волне 3.

Приоритет блока: T-1 + T-2 (high) — волна 1; T-3/T-4 — волна 2; T-5 — волна 3.

09 — Наблюдаемость

В ТЗ пункт 8 оставлен без расшифровки. Для **библиотеки** наблюдаемость трактуем как: предсказуемое логирование, диагностируемость в потребителе, версионирование/changelog, и хуки для интеграции с мониторингом приложения-потребителя (а не встроенный SDK телеметрии — это было бы навязыванием).

О-1. Бессистемное логирование (`console.*`)

- **Где:** 3 вхождения `console.*`, в т.ч. `validate/index.ts` (`console.error('Form dont have vatNumber or countryId')`), линт-правило `no-console: warn`.
- **Серьёзность:** medium.
- **Описание:** библиотека пишет напрямую в `console` потребителя, без уровня/префикса/возможности отключить. Шум в проде, нет контроля.
- **Решение:** единый dev-only логгер с префиксом `[invoicebox/ui]`, отключаемый в production-сборке (`process.env.NODE_ENV`), без прямых `console` в рантайм-коде.
- **Эффект:** чистый прод-лог потребителя, управляемая диагностика.

О-2. Нет механизма репортинга ошибок наружу

- **Где:** компоненты с асинхронной логикой / валидацией.
- **Серьёзность:** medium.
- **Описание:** когда внутри компонента что-то идёт не так (см. `02-resilience`), у потребителя нет колбэка/хука узнать об этом и отправить в свой Sentry/мониторинг.
- **Решение:** опциональный проп `onError` /контекст `onError` на составных компонентах и в `ErrorBoundary` (R-1); документировать.
- **Эффект:** потребитель видит сбои библиотеки в своём мониторинге.

О-3. Нет CHANGELOG / прозрачности версий

- **Где:** `release.sh` бампит `prerelease --preid=alpha` без changelog; `RELEASE.md` описывает процесс, но истории изменений API нет.
- **Серьёзность:** low/medium.
- **Описание:** потребитель не понимает, что изменилось между alpha-версиями (а их уже 292) — нет наблюдаемости эволюции API.
- **Решение:** автогенерация CHANGELOG (`changesets/conventional-changelog`) в релиз-пайплайне.
- **Эффект:** прозрачность изменений, безопасные апгрейды у потребителей.

О-4. Версия библиотеки недоступна в рантайме

- **Где:** нет экспорта `version`.
- **Серьёзность:** low.
- **Описание:** при дебаге у потребителя нельзя программно узнать версию UI-kit.
- **Решение:** экспортировать `version` (из `package.json` через сборку).
- **Эффект:** проще диагностика инцидентов («какая версия установлена»).

О-5. Наблюдаемость сборки/релиза

- **Где:** процесс релиза (см. PL-5).
 - **Серьёзность:** low.
 - **Описание:** нет следов сборки (логов CI, артефактов, size-отчёта) — релиз непрозрачен.
 - **Решение:** CI-артефакты: отчёт `size-limit`, `coverage`, `npm audit`, лог сборки на каждый релиз.
 - **Эффект:** аудируемая история качества релизов.
-

Приоритет блока: О-1/О-2 — волна 2; О-3 — волна 2–3; О-4/О-5 — волна 3.

10 — План рефакторинга (волнами)

Принцип очередности: **сначала риски и измеримость** — сперва ставим «сетку безопасности» (CI, тесты, измерения), затем закрываем высокий риск, затем качество и доступность. Бизнес-логику и пользовательские пути **не меняем без согласования** (отдельное продуктивное решение).

Каждая волна = отдельная ветка/PR. Документацию синхронизируем в том же PR, что и код. После каждой волны — повторный замер метрик из `01-BASELINE-METRICS.md`.

Волна 0 — «Сетка безопасности» (риск + измеримость, без правок бизнес-логики)

Цель: начать измерять и не дать регрессий. Минимум изменений поведения.

1. **CI** (T-1, PL-5): GitHub Actions — lint + typecheck + test + build + `npm audit`.
2. **Jest-конфиг** (T-4): `testPathIgnorePatterns: /dist/`, `coverage`, `coverageThreshold`, `jsdom`.
3. **Измерения** (P-3): `size-limit` бюджеты на ключевые экспорты; baseline зафиксирован.
4. **Lint-гейты дешево** (AR-2, AR-6): `max-lines` (warn), `eslint-plugin-jsx-a11y`, `@storybook/addon-a11y` — пока как warn, чтобы увидеть масштаб А-блока.
5. **Удалить мёртвое** (P-2, AR-6): убрать `motion`; заменить shim-полифилы `object.entries/fromentries` нативными.

Эффект/метрика: появляется CI-гейт; coverage и size измеряются; −1..3 зависимости. **Риск:** минимальный (инфраструктура).

Волна 1 — Высокий риск: безопасность + платформа + критич. тесты

1. **S-1:** `isSafeUrl` + `rel="noopener noreferrer"` в `AppCardLink`.
2. **S-2:** правый якорь и JSDoc для `Validate.website`, ReDoS-чек.
3. **PL-2 + S-3:** миграция Storybook 7 → 8/9 (Vite), выпил `react-scripts` /CRA-пресета → падение `npm audit` на десятки CVE.
4. **PL-3:** `engines.node >=18`, `.nvmrc`, CI-матрица.
5. **T-2:** unit-тесты на весь `validate/` + phone/currency/date-логику (ROI максимум).

Эффект/метрика: `npm audit critical`→0, coverage `validate/` ≥80%, закрыт XSS-вектор. **Риск:** средний (Storybook-миграция); бизнес-логика не трогается.

Волна 2 — Производительность сборки + типы + наблюдаемость

1. **P-1 + PL-4:** переход на `tsup` / `rollup`: ESM+CJS, `exports -map` с подпутями, `sideEffects`, минификация; убрать `dist` из git, собирать в CI.
2. **PL-1:** апгрейд TS 5 / ESLint 9 / typescript-eslint 8.
3. **AR-1:** убрать `any` /unsafe assertions, включить `no-explicit-any` / `no-unsafe-*`.
4. **O-1/O-2:** единый dev-логгер + опциональный `onError`.
5. **T-3:** RTL + `jest-axe` + Storybook play-тесты на ключевые компоненты.

Эффект/метрика: tree-shakable пакет (замер size до/после), типобезопасный API, coverage ≥50%.
Риск: средний (смена сборщика — проверить совместимость экспортов у потребителей).

Волна 3 — Доступность (по компонентам) + качество + резилентность

Самый объёмный блок; вести покомпонентно, каждый компонент — отдельный небольшой PR со stories + a11y-тестом. 1. **A-1/A-2/A-3** (critical/high): клавиатура+ARIA для Select/Dropdown/Autocomplete/ Toggle; label↔control+ошибки; фокус-трап в Drawer. Рассмотреть Radix/React Aria как базу примитивов. 2. **A-4/A-5/A-6**: контраст палитры, alt у SVG, prefers-reduced-motion, rem /reflow. 3. **P-5/P-6**: Skeleton с резервированием размеров (CLS), anti-flicker загрузчик. 4. **R-1/R-2/R-4/R-5**: ErrorBoundary, guard-парсинг, безопасный useLoadingSubmit, fallback палитры. 5. **AR-3/AR-5, O-3/O-4**: ререндеры, стайлгайд API, CHANGELOG, экспорт version.

Эффект/метрика: jest-axe /addon-a11y без нарушений на покрытых компонентах; CLS≈0; coverage ≥70%. **Риск:** изменения поведения компонентов — **каждый a11y-PR, меняющий разметку/фокус, согласуется** (может затронуть верстку потребителей).

Что требует явного согласования перед стартом

- 1. **Storybook-миграция 7→8/9** (волна 1) — затрагивает dev-процесс команды.
- 2. **Смена сборщика и exports -map** (волна 2) — меняет способ импорта у потребителей (обратная совместимость barrel-импорта сохраняется, подпути добавляются).
- 3. **A11y-изменения разметки/ролей** (волна 3) — могут влиять на стили/селекторы потребителей; продуктивное решение по приоритету компонентов.
- 4. **Удаление motion** — подтвердить, что не планируется к использованию.

Сводная таблица приоритетов

ID	Блок	Severity	Волна
T-1	CI	high	0
P-2	мёртвый motion	high	0
S-1	href/_blank XSS	high	1
PL-2	CVE через CRA-пресет	high	1
PL-5	ручной релиз	high	1
T-2	тесты критич. логики	high	1
S-2	website regex	medium	1
P-1	CJS/tree-shaking	high	2
A-1	клавиатура	critical	3
A-2	label/ошибки	high	3
A-3	фокус-трап	high	3
(остальные medium/low)	—	—	2–3